LAB 4 - Linear_Regression_Gradient_Descent

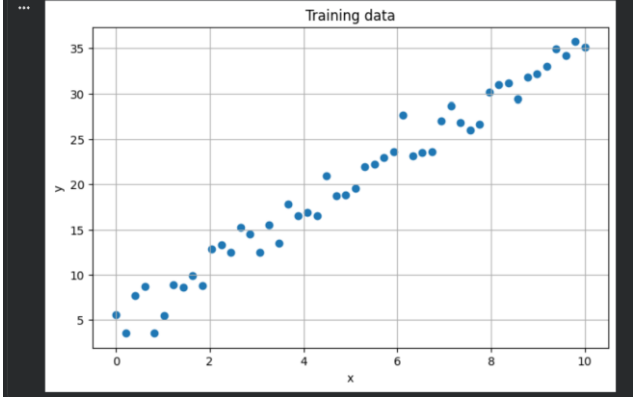
Họ và tên: Nguyễn Hữu Hạo MSSV:2474802010108

3.0 – Import thư viện

- Chuẩn bị đầy đủ các thư viện cần thiết cho quá trình xây dựng, huấn luyện và đánh giá mô hình.

3.1 – Tạo dữ liệu

```
rng = np.random.default_rng(42)
x = np.linspace(0, 10, 50)
noise = rng.normal(0, 2, size=x.size)
y = 3*x + 5 + noise
plt.figure(figsize=(8, 5))
plt.scatter(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Training data")
plt.grid(True)
plt.show()
```



3.tạo ra **50 mẫu dữ liệu** với: $x \in [0, 10]$

Dữ liệu được xây dựng gần theo phương trình: y=3x+5+ϵ

Seed 42 giúp kết quả có thể được tái tạo giống nhau khi chạy lại.

3.2 – Xây dựng hàm dự đoán và MSE

Hàm predict() thực hiện công thức Linear Regression: $\hat{y} = wx + b$

Hàm mse_loss() tính Mean Squared Error: $MSE = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$

MSE được sử dụng làm hàm loss cần tối thiểu hóa.

3.3 – Tính MSE ban đầu

```
w0, b0 = 0.0, 0.0
y_hat0 = predict(x, w0, b0)
print("Initial MSE =", mse_loss(y, y_hat0))

Initial MSE = 492.4133694116765
```

Ban đầu chọn: $w_0 = 0,\ b_0 = 0$

Do đó: $\hat{y} = 0$ (cho tất cả các mẫu)

Kết quả: $MSE_{initial}$=492.4134

3.4 – Tính Gradient

3.này tính gradient của MSE theo hai tham số $w$và

3.5 – Cài đặt Batch Gradient Descent

Hàm linear_regression_gd() thực hiện toàn bộ quá trình huấn luyện:

1. Khởi tạo w, b.
2. Tính giá trị dự đoán.
3. Tính MSE.
4. Tính gradient.
5. Cập nhật w, b.
6. Lưu MSE vào history.
7. Kiểm tra điều kiện dừng.
8. Trả về kết quả cuối cùng.

3.6 – Huấn luyện với learning rate = 0.01

```
result = linear_regression_gd(
 x, y,
 lr=0.01,
 epochs=1000,
 w0=0.0,
 b0=0.0
)
print("w =", result["w"])
print("b =", result["b"])
print("epochs =", result["epochs"])
print("runtime =", result["runtime"])
print("final MSE =", result["history"][-1])
```

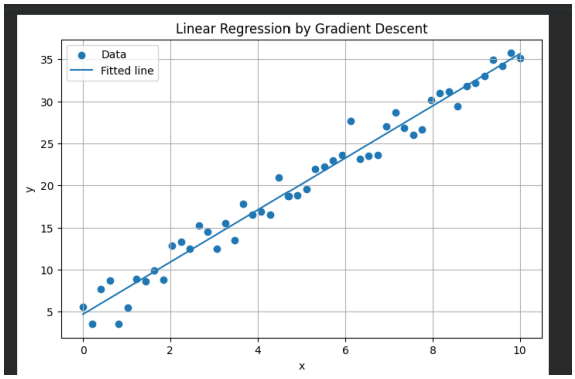
```
w = 3.09446474227027
b = 4.7033723560562315
epochs = 1000
runtime = 0.0276202730000250l
final MSE = 2.242972046811748
```

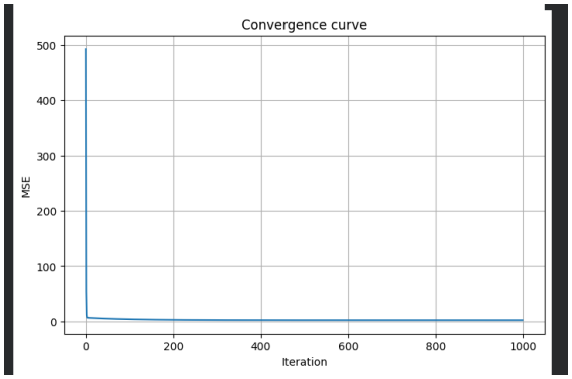
Mô hình được huấn luyện với:$\eta = 0.01$

và bắt đầu từ: $w_0 = 0,\ b_0 = 0$

Sau 1000 iteration, mô hình tìm được:= $\hat{y}$≈ 3.0945x+4.7034

3.7 – Fitted Line và Convergence Curve





3.này tạo hai biểu đồ:

Biểu đồ 1: dữ liệu thực tế và đường hồi quy: $\hat{y} = wx + b$

Biểu đồ 2: MSE theo iteration, gọi là convergence curve.

3.8 – Thí nghiệm Learning Rate

```
learning_rates = [0.0001, 0.001, 0.01, 0.02, 0.03]
results_lr = []
for lr in learning_rates:
 r = linear_regression_gd(x, y, lr=lr, epochs=500)
 final_loss = r["history"][-1]
 status = "OK" if np.isfinite(final_loss) and final_loss < 1e6 else "Diverged"
 results_lr.append([
 lr, r["w"], r["b"],
 final_loss, r["epochs"],
 r["runtime"], status
 ])
for row in results_lr:
 print(row)
```

```
[0.0001, np.float64(3.5819667630764185), np.float64(0.6420740790405083), np.float64(7.003777860382575), 500, 0.01516534600000341, 'OK']
[0.001, np.float64(3.5758341755943186), np.float64(1.4857555125389066), np.float64(4.957091071654243), 500, 0.014425374999973428, 'OK']
[0.01, np.float64(3.1404095076219516), np.float64(4.396263843066162), np.float64(2.271782787220718), 500, 0.015137424999920768, 'OK']
[0.02, np.float64(3.0944138437430757), np.float64(4.703712576990662), np.float64(2.2429691950768866), 500, 0.01773159399999713, 'OK']
[0.03, np.float64(-201266020788834.5), np.float64(-30110269521272.86), np.float64(1.2561696615953654e+30), 500, 0.013832336999939798, 'Diverged']
```

Thử 5 giá trị: 0.0001, 0.001, 0.01, 0.02, 0.03

- 0.0001: quá nhỏ nên sau 500 iteration vẫn chưa tiến gần nghiệm tối ưu.
- 0.001: tốt hơn nhưng vẫn còn chậm.
- 0.01: hội tụ khá tốt.
- 0.02: cho kết quả tốt nhất trong thí nghiệm.
- 0.03: quá lớn làm thuật toán phân kỳ, MSE tăng cực mạnh.

Điều này phù hợp với lý thuyết của Lab: learning rate quá nhỏ làm loss giảm chậm, còn quá lớn có thể khiến loss dao động hoặc phân kỳ.

3.9 – Đánh giá mô hình

MSE: sai số bình phương trung bình.

RMSE: căn bậc hai của MSE.

R^2: mức độ mô hình giải thích được sự biến thiên của dữ liệu.

```
mse = mean_squared_error(y, y_hat)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_hat)
print("MSE =", mse)
print("RMSE =", rmse)
print("R2 =", r2)
```

```
MSE = 2.2429701826717627
RMSE = 1.497654894383804
R2 = 0.9736379221621626
```

3.10 – So sánh với Scikit-learn

```
model = LinearRegression()
model.fit(x.reshape(-1, 1), y)
y_sklearn = model.predict(x.reshape(-1, 1))
print("sklearn w =", model.coef_[0])
print("sklearn b =", model.intercept_)
print("sklearn MSE =", mean_squared_error(y, y_sklearn))
print("sklearn R2 =", r2_score(y, y_sklearn))
```

```
sklearn w = 3.0904713914955027
sklearn b = 4.730065103508512
sklearn MSE = 2.2427866291632803
sklearn R2 = 0.9736400795033148
```

Hai phương pháp cho kết quả rất gần nhau.

Gradient Descent tự cài đặt cho kết quả hơi khác Scikit-learn vì Gradient Descent tìm nghiệm thông qua quá trình lặp với learning rate và số iteration hữu hạn, trong khi LinearRegression của Scikit-learn sử dụng phương pháp giải bài toán Linear Regression trực tiếp.

3.11 – Ảnh hưởng của điểm khởi tạo

```
initializations = [(0, 0), (10, -20), (-5, 20)]
for w0, b0 in initializations:
 r = linear_regression_gd(
 x, y, lr=0.01, epochs=1500,
 w0=w0, b0=b0
 )
 print(
 "init =", (w0, b0),
 "-> w =", r["w"],
 "b =", r["b"],
 "MSE =", r["history"][-1]
 )
```

```
init = (0, 0) -> w = 3.0907907237340257 b = 4.7279305915968255 MSE = 2.2427878148274054
init = (10, -20) -> w = 3.0923991666921853 b = 4.717179279204028 MSE = 2.242829839626795
init = (-5, 20) -> w = 3.0892382515509595 b = 4.738307778637623 MSE = 2.2428043099212966
```

Mặc dù điểm khởi tạo khác nhau nhưng cả ba trường hợp đều hội tụ về vùng nghiệm gần giống nhau: $w \approx 3.09$, $b \approx 4.73$

MSE đều xấp xỉ: 2.243

Cho thấy với learning rate ổn định, bài toán Linear Regression có MSE là hàm lồi nên các điểm khởi tạo khác nhau thường vẫn tiến về cùng vùng nghiệm tối ưu.

3.12. Câu hỏi phân tích tại lớp

1. Vì sao w và b được gọi là biến quyết định?

w và b là hai tham số cần tìm của mô hình $\hat{y} = wx + b$. Việc thay đổi w và b sẽ làm thay đổi kết quả dự đoán, nên chúng là các biến quyết định cần tối ưu để MSE nhỏ nhất.

2. Nếu learning rate rất nhỏ, vì sao MSE sau 500 iteration vẫn cao?

Learning rate quá nhỏ làm mỗi bước cập nhật w và b rất ít, khiến Gradient Descent hội tụ chậm. Sau 500 iteration, mô hình có thể chưa đến gần nghiệm tối ưu nên MSE vẫn cao.

3. Vì sao $\eta = 0.03$ có thể làm loss tăng mạnh?

$\eta = 0.03$ quá lớn đối với dữ liệu hiện tại. Mỗi bước cập nhật quá mạnh, có thể vượt qua nghiệm tối ưu và làm loss tăng liên tục, dẫn đến phân kỳ.

4. Gradient Descent và scikit-learn khác nhau như thế nào?

Gradient Descent tự tính gradient và cập nhật w, b qua nhiều iteration. Scikit-learn sử dụng phương pháp Linear Regression có sẵn để tìm nghiệm. Vì vậy kết quả gần nhau nhưng có thể không hoàn toàn giống nhau.

5. Nếu x từ 0 đến 1.000.000, việc chọn learning rate có khó hơn không? Vì sao scaling hữu ích?

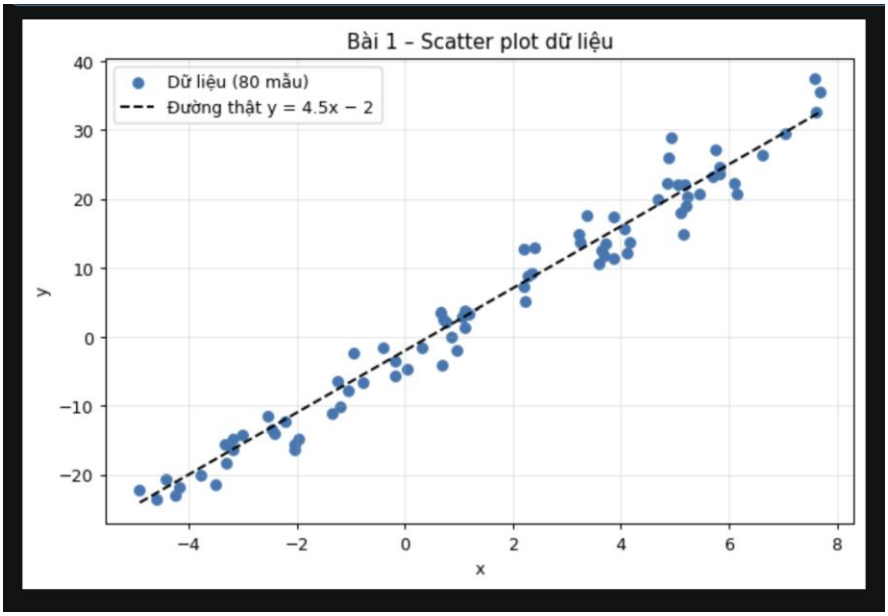
Có. x quá lớn làm gradient lớn, khiến learning rate khó lựa chọn và dễ gây dao động hoặc phân kỳ. Feature scaling đưa dữ liệu về phạm vi phù hợp hơn, giúp Gradient Descent ổn định và hội tụ nhanh hơn.

6. Khi nào convergence curve cho thấy mô hình chưa ổn định?

Khi MSE dao động mạnh, tăng liên tục, tăng lên giá trị rất lớn hoặc xuất hiện NaN/Inf.

BÀI TẬP

1.1 - Sinh dữ liệu và vẽ Scatter Plot



- Tạo 80 mẫu dữ liệu theo mô hình (y=4.5x-2+\varepsilon), sau đó biểu diễn các điểm dữ liệu trên đồ thị.
- Các điểm dữ liệu có xu hướng tăng theo (x), phù hợp với mô hình tuyến tính. Do có nhiễu ngẫu nhiên nên các điểm không nằm hoàn toàn trên đường thẳng thật.

1.2 - Ước lượng (w,b) bằng Gradient Descent

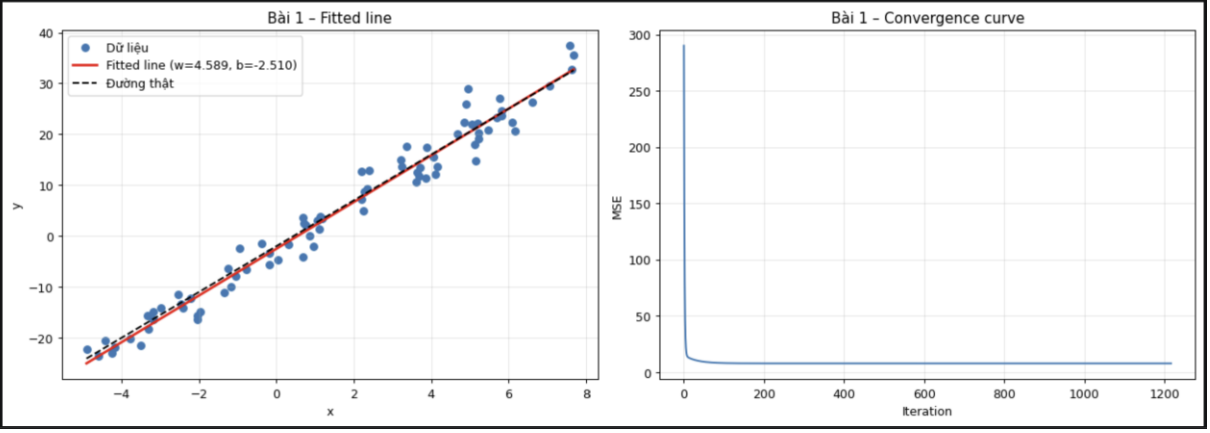
```
res1 = linear_regression_gd(x_b1, y_b1, lr=0.01, epochs=3000, w0=0.0, b0=0.0)
w1, b1 = res1["w"], res1["b"]
print(f"GD: w = {w1:.4f}, b = {b1:.4f}")
print(f"Iterations = {res1['epochs']}, runtime = {res1['runtime']:.4f}s, final MSE = {res1['history'][-1]:.4f}")
print(f"Thật: w = {TRUE_W}, b = {TRUE_B}")
```

```
GD      : w = 4.5887, b = -2.5103
Iterations = 1217, runtime = 0.0171s, final MSE = 7.7325
Thật    : w = 4.5, b = -2.0
```

- Sử dụng Gradient Descent tự cài đặt để tìm tham số (w,b) làm MSE nhỏ nhất.
- Mô hình thu được (w\approx4.5887), (b\approx-2.5103), sau 1217 iteration với MSE ≈ 7.7325. Kết quả khá gần tham số thật (4.5,-2).

1.3 - Fitted Line và Convergence Curve

```
y1_hat = predict(x_b1, w1, b1)
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].scatter(x_b1, y_b1, label="Dữ liệu")
ax[0].plot(x_b1, y1_hat, "r", lw=2, label=f"Fitted line (w={w1:.3f}, b={b1:.3f})")
ax[0].plot(x_b1, TRUE_W*x_b1 + TRUE_B, "k--", label="Đường thật")
ax[0].set_xlabel("x"); ax[0].set_ylabel("y")
ax[0].set_title("Bài 1 - Fitted line"); ax[0].grid(True, alpha=0.3); ax[0].legend()
ax[1].plot(res1["history"])
ax[1].set_xlabel("Iteration"); ax[1].set_ylabel("MSE")
ax[1].set_title("Bài 1 - Convergence curve"); ax[1].grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```



- Vẽ đường hồi quy sau huấn luyện và theo dõi sự thay đổi của MSE qua từng iteration.
- Fitted line bám khá sát các điểm dữ liệu. Convergence curve giảm dần rồi ổn định, cho thấy Gradient Descent đã hội tụ.

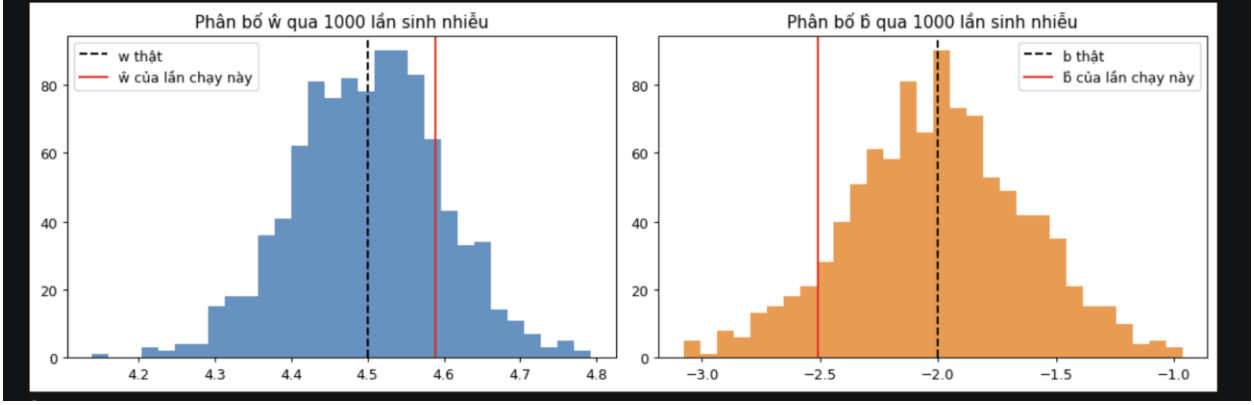
1.4 và 1.5 - Tính chỉ số và so sánh với scikit-learn

```
                  Phương pháp       w        b     MSE    RMSE       R²
0  Gradient Descent (tự viết) 4.58869 -2.51025 7.73252 2.78074 0.971084
1                scikit-learn 4.58869 -2.51025 7.73252 2.78074 0.971084
2         Closed-form (lstsq) 4.58869 -2.51025     NaN     NaN      NaN
3                Tham số thật     4.5       -2     NaN     NaN      NaN
|w_GD - w_sklearn| = 6.03e-10
|b_GD - b_sklearn| = 5.33e-09
Nhiễu σ² = 9.0  →  MSE huấn luyện (7.733) xấp xỉ σ² (không thể thấp hơn nhiều vì nhiễu là ngẫu nhiên)
```

- Tính MSE, RMSE, (R^2) và kiểm tra kết quả Gradient Descent với LinearRegression của scikit-learn.
- GD và scikit-learn cho kết quả gần như giống hệt nhau: (w\approx4.5887), (b\approx-2.5103), (R^2\approx0.971). Điều này cho thấy code Gradient Descent tự viết hoạt động đúng.

1.6 - Lặp lại 1000 lần

```
ŵ: trung bình = 4.5011 (thật 4.5),  độ lệch chuẩn = 0.0963  (lý thuyết σ/√Sxx = 0.0955)
b̂: trung bình = -2.0032 (thật -2),   độ lệch chuẩn = 0.3759  (lý thuyết = 0.3680)
```



- Giữ nguyên x, mỗi lần sinh nhiễu mới, rồi ước lượng lại (ŵ, b̂) bằng closed-form để chạy nhanh.
- se_w, se_b là sai số chuẩn theo lý thuyết, so sánh với độ lệch chuẩn thực nghiệm (0.096 và 0.37).
- Histogram cho thấy ŵ và b̂ phân bố quanh giá trị thật.

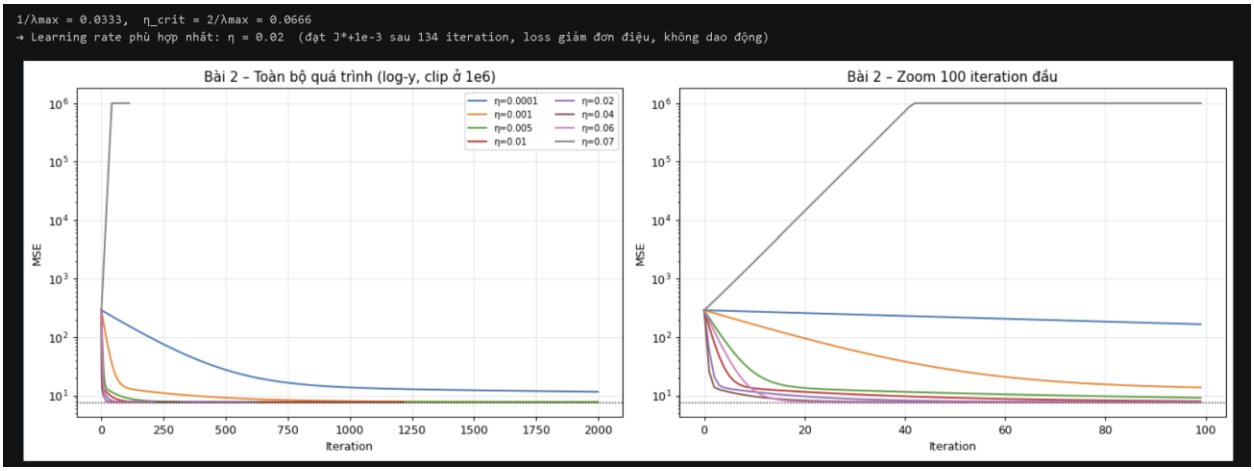
Nhận xét: ŵ, b̂ là biến ngẫu nhiên, trung bình đúng nhưng mỗi mẫu lệch một ít. Lần chạy của bạn chỉ lệch khoảng 1 độ lệch chuẩn, nên bình thường.

2.1 – Chạy nhiều Learning Rate và lập bảng

```
Trị riêng của Hessian: [ 1.64168566 30.04947944]
Ngưỡng ổn định η_crit = 2/λmax = 0.0666   |   MSE tối ưu J* = 7.732519
       η  Iterations (dừng)       w         b    MSE cuối  Runtime (s)  Iter đạt J*+1e-3  Loss giảm đơn điệu  η > 1/λmax (tham số dao động dấu)             Trạng thái
0 0.0001              2000 4.33494 -0.357831     11.5911    0.0270817               NaN                True                              False  Chưa hội tụ (quá chậm)
1  0.001              2000 4.57604   -2.3984     7.74295     0.026179               NaN                True                              False  Chưa hội tụ (quá chậm)
2  0.005              2000 4.58869  -2.51025     7.73252    0.0241913               541                True                              False                 Hội tụ
3   0.01              1217 4.58869  -2.51025     7.73252    0.0148468               270                True                              False                 Hội tụ
4   0.02               625 4.58869  -2.51025     7.73252    0.0075566               134                True                              False                 Hội tụ
5   0.04               318 4.58869  -2.51025     7.73252   0.00375503                66                True                               True                 Hội tụ
6   0.06               213 4.58869  -2.51025     7.73252   0.00250967                43                True                               True                 Hội tụ
7   0.07               113 -261424  -29550.1 1.03998e+12   0.00126074               NaN               False                               True                Phân kỳ
```

- Thử 8 learning rate khác nhau để đánh giá tốc độ hội tụ, MSE, số iteration và trạng thái của Gradient Descent.
- Learning rate quá nhỏ như 0.0001 và 0.001 hội tụ rất chậm. Các giá trị từ 0.005 đến 0.06 hội tụ tốt, trong khi (\eta=0.07) bị phân kỳ.

2.2 – Vẽ convergence curve và chọn Learning Rate



- Trực quan hóa ảnh hưởng của từng learning rate và chọn giá trị phù hợp nhất dựa trên tốc độ và độ ổn định.
- (\eta=0.02) được chọn là phù hợp nhất vì đạt MSE tối ưu sau khoảng **134 iteration**, loss giảm ổn định và không dao động.

3.1 – Sinh dữ liệu và phân tích Hessian

```
Closed-form: w = 2.0061, b = 4.3139, MSE* = 301.2447
A (x gốc)  : λ = [5.03758271e-01 6.68343205e+05],  condition number κ = 1.327e+06,  η_crit = 2.992e-06
B (z-score): λ = [2. 2.],  condition number κ = 1.000,  η_crit = 1.000
```

- Tạo dữ liệu (x\in[0,1000]), sau đó so sánh độ ổn định của Gradient Descent trước và sau khi chuẩn hóa z-score.

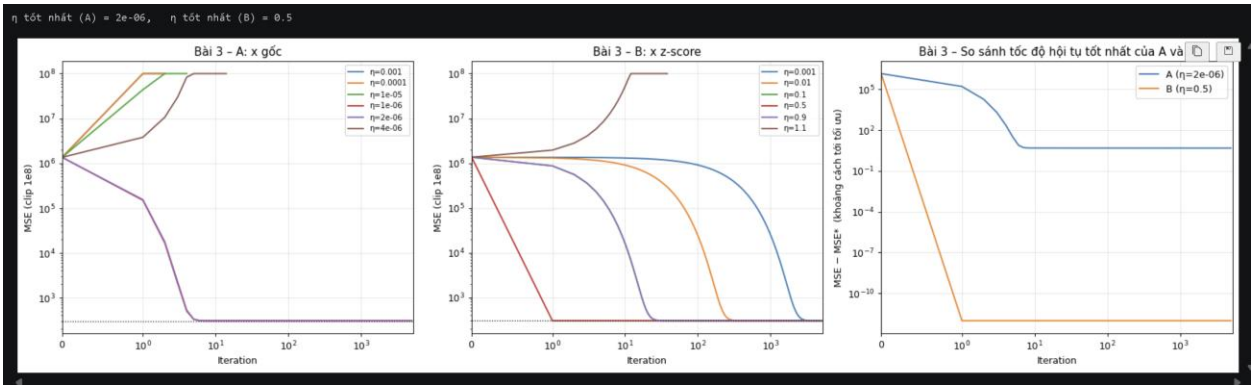
- Với (x) gốc, condition number rất lớn (\approx1.327\times10^6) và learning rate ổn định phải rất nhỏ. Sau z-score, condition number giảm còn 1 và ngưỡng learning rate tăng lên đáng kể.

3.2 – Thử nhiều Learning Rate cho A và B

```
=== Trường hợp A: x gốc ===
       η  Iterations  w (thang gốc)  b (thang gốc)    MSE cuối  Iter đạt J*(1+1e-3)  Runtime (s)   Trạng thái
0  0.001           3        -896278       -1341.04 2.68447e+17                   NaN    6.462e-05      Phân kỳ
1 0.0001           3       -8720.67       -13.0478 2.54256e+13                   NaN   3.4598e-05      Phân kỳ
2  1e-05           5       -2097.84       -3.13878 1.47349e+12                   NaN   8.6976e-05      Phân kỳ
3  1e-06        5000        2.01253      0.0138559     305.902                   NaN    0.0672426  Chưa hội tụ
4  2e-06        5000        2.01251      0.0246733     305.879                   NaN    0.0690339  Chưa hội tụ
5  4e-06          15        -2714.7       -4.06172 2.46636e+12                   NaN  0.000203838      Phân kỳ
=== Trường hợp B: x chuẩn hóa z-score ===
      η  Iterations  w (thang gốc)  b (thang gốc)    MSE cuối  Iter đạt J*(1+1e-3)  Runtime (s) Trạng thái
0 0.001        5000          2.006        4.31374     301.247                 3826    0.0666454     Hội tụ
1  0.01        5000        2.00609        4.31393     301.245                  380    0.0689311     Hội tụ
2   0.1        5000        2.00609        4.31393     301.245                   35    0.0638234     Hội tụ
3   0.5        5000        2.00609        4.31393     301.245                    1    0.0639207     Hội tụ
4   0.9        5000        2.00609        4.31393     301.245                   35    0.0626537     Hội tụ
5   1.1          39       -2045.56        -4398.8 1.41006e+12                  NaN  0.000549089    Phân kỳ
```

- So sánh trực tiếp khả năng hội tụ của dữ liệu gốc và dữ liệu đã chuẩn hóa.
- Trường hợp A rất dễ phân kỳ nếu learning rate lớn. Trường hợp B ổn định hơn nhiều, các learning rate từ 0.001 đến 0.9 đều hội tụ trong thí nghiệm.

3.3 – Chọn Learning Rate tốt nhất và vẽ đồ thị



- Chọn learning rate có tốc độ hội tụ tốt nhất và trực quan hóa sự khác biệt giữa hai trường hợp.
- Learning rate tốt nhất của A là (2\times10^{-6}), còn B là **0.5**. Dữ liệu chuẩn hóa hội tụ nhanh và ổn định hơn rất nhiều.

3.4 – Chạy 100.000 iteration

```
A, η=2e-06, 100.000 iteration: w = 2.0119, b = 0.4162, MSE = 305.0714  (tối ưu: w = 2.0061, b = 4.3139, MSE* = 301.2447)
B, η=0.5, chỉ 50 iteration:      w = 2.0061, b = 4.3139, MSE = 301.2447
```

- Kiểm tra xem nếu tăng rất nhiều iteration thì dữ liệu gốc có thể bắt kịp dữ liệu đã chuẩn hóa hay không.
- Sau 100.000 iteration, trường hợp A vẫn chưa đạt nghiệm tối ưu, trong khi B chỉ cần khoảng 50 iteration để đạt gần nghiệm tối ưu. Điều này thể hiện rõ lợi ích của feature scaling.

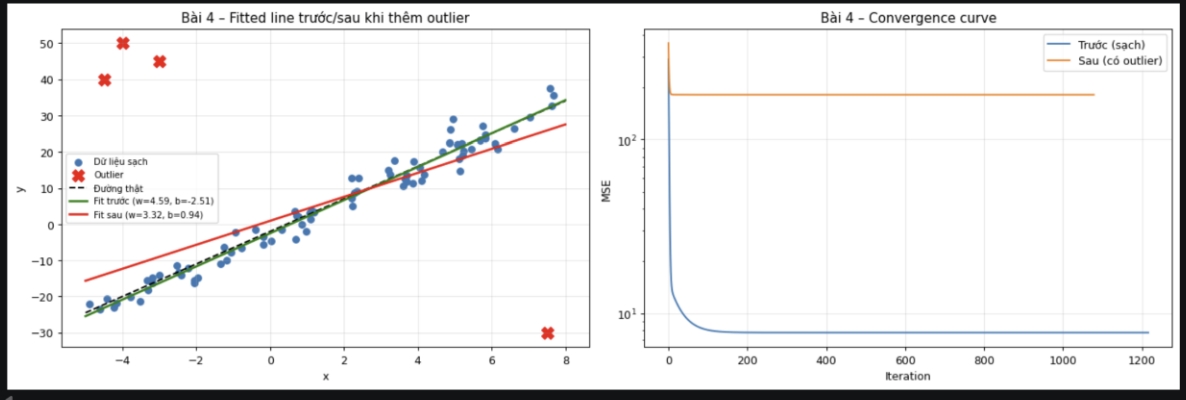
4.1 – Thêm Outlier và huấn luyện lại

```
GD có outlier      : w = 3.3172, b = 0.9367
Closed-form (đối chiếu): w = 3.3172, b = 0.9367
                                Trường hợp       w        b     MSE    RMSE       R²
0             Trước (dữ liệu sạch, 80 mẫu) 4.58869 -2.51025 7.73252 2.78074 0.971084
1               Sau (có 4 outlier, 84 mẫu) 3.31721 0.936721 181.082 13.4567 0.445278
2  Mô hình 'sau' chấm lại trên 80 mẫu sạch 3.31721 0.936721 29.7198 5.45159 0.888863
4 outlier chiếm 4.8% số mẫu nhưng đóng góp 84.4% tổng bình phương sai số của mô hình sau.
```

- Thêm 4 điểm bất thường vào dữ liệu và đánh giá mức độ ảnh hưởng của chúng đến mô hình.
- Sau khi thêm outlier, (w) giảm từ 4.59 xuống 3.32 và (b) tăng từ -2.51 lên 0.94. MSE tăng mạnh từ 7.73 lên 181.08, còn (R^2) giảm từ 0.971 xuống 0.445.

4.2 – Vẽ Fitted Line, Convergence Curve và kiểm tra MSE

```
|sai số|  Đóng góp MAE  Đóng góp MSE (e²)
       1              1                 1
       2              2                 4
       5              5                25
      10             10               100
      20             20               400
      40             40              1600
```



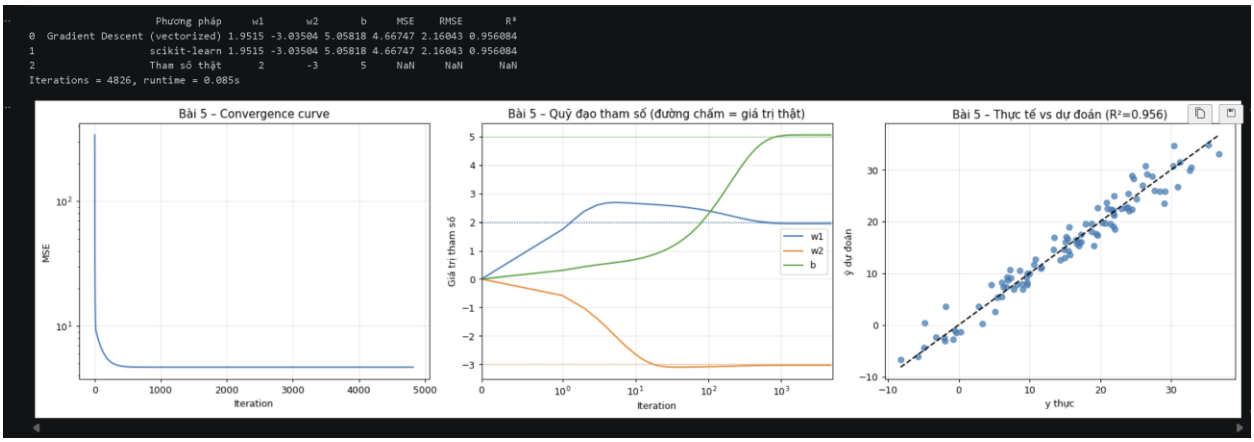
- Quan sát trực quan sự thay đổi của đường hồi quy và minh họa mức độ nhạy của MSE với sai số lớn.
- Đường hồi quy bị kéo lệch về phía các outlier. Sai số càng lớn thì đóng góp vào MSE càng nhanh do bị bình phương, nên MSE rất nhạy với outlier.

5.1 – Xây dựng Gradient Descent cho nhiều biến

```
Kiểm tra gradient (giải tích vs sai phân hữu hạn):
  dw: [-126.32884516   36.09169446] [-126.32884518   36.09169444] | db: -22.583280037667397 -22.58328002824328
  Sai lệch tối đa = 2.6483519377507037e-08
```

- Mở rộng mô hình từ một biến sang hai biến với (\hat y=w_1x_1+w_2x_2+b), sử dụng NumPy vectorization.
- Gradient giải tích gần như trùng với sai phân hữu hạn, sai lệch tối đa chỉ khoảng (2.65\times10^{-8}). Điều này chứng minh công thức gradient được cài đặt chính xác.

5.2 – Huấn luyện, đánh giá và so sánh



- Huấn luyện mô hình hai biến, tính MSE, RMSE, (R^2) và so sánh với scikit-learn.
- GD thu được (w_1\approx1.9515), (w_2\approx-3.0350), (b\approx5.0582), gần tham số thật (2,-3,5). Kết quả gần như trùng với scikit-learn, với (R^2\approx0.956), cho thấy mô hình hoạt động tốt.